INSTRUCTION REFERENCE

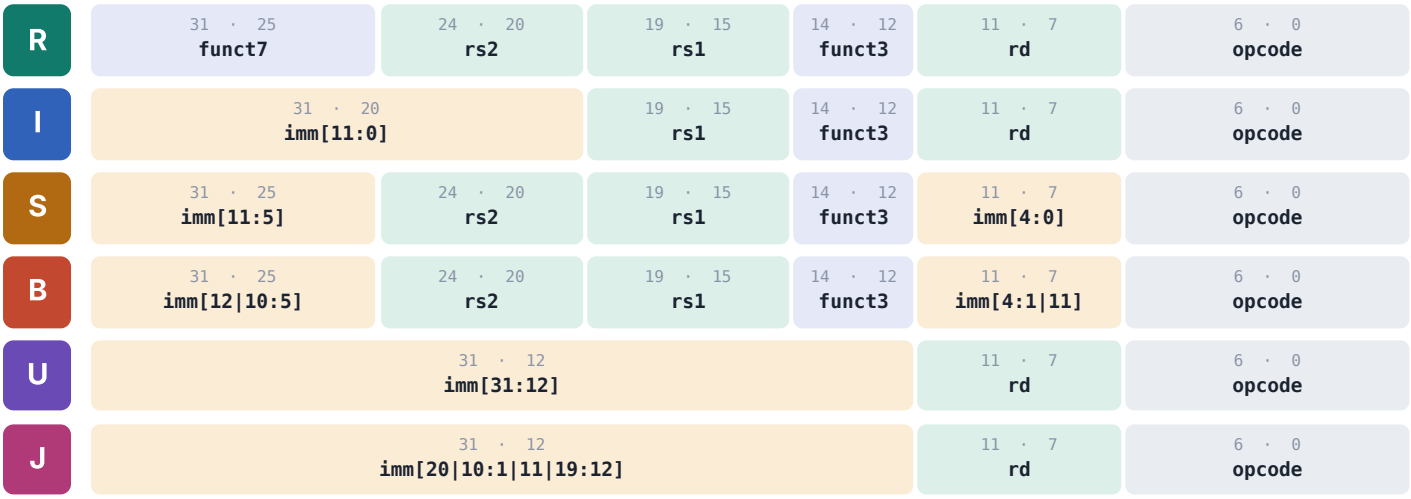
RV32I on arielhuna/riscv-cpu

Every instruction this single-cycle SystemVerilog core executes: encoding, behaviour, the control signals `rtl/control.sv` drives for it, and a machine-code example produced by `rvasm.py`.

37	9	6	32 × 32	1 CPI
instructions implemented	opcodes decoded	encoding formats	register file, x0 = 0	single cycle, Harvard

Encoding formats

32-bit, little-endian, bit 31 on the left



opcode · register · function · immediate

rs1, rs2, rd and the sign bit (instr[31]) sit in the same place in every format.

Immediate generator

rtl/immgen.sv · i = instr

FMT	USED BY	IMM[31:0]	RANGE
I	ADDI..., loads, JALR	`{{20{i[31]}}, i[31:20]}`	−2048 … 2047
S	stores	`{{20{i[31]}}, i[31:25], i[11:7]}`	−2048 … 2047
B	branches	`{{20{i[31]}}, i[7], i[30:25], i[11:8], 1'b0}`	±4 KiB, even
U	LUI, AUIPC	`{i[31:12], 12'b0}`	upper 20 bits
J	JAL	`{{12{i[31]}}, i[19:12], i[20], i[30:21], 1'b0}`	±1 MiB, even
R	R-type	`32'b0 (unused)`	—

Registers

ABI names accepted by rvasm.py

REG	ABI	ROLE	REG	ABI	ROLE
x0	zero	hard-wired 0	**x8**	s0 / fp	saved / frame ptr
x1	ra	return address	**x9**	s1	saved
x2	sp	stack pointer	**x10–x11**	a0–a1	args / return values
x3	gp	global pointer	**x12–x17**	a2–a7	arguments
x4	tp	thread pointer	**x18–x27**	s2–s11	saved
x5–x7	t0–t2	temporaries	**x28–x31**	t3–t6	temporaries

Arithmetic and logic

ALU instructions, result written to rd

R Register–register

opcode **0110011** · OP_R

`alu_src 0` `alu_a_sel 0` `wb_sel 0` `reg_we 1` `mem_we 0` `branch 0` `jump 0` `jalr 0`

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
ADD	`ADD rd, rs1, rs2`	rd = rs1 + rs2	000	0000000	ALU_ADD	`ADD x5, x6, x7`	**007302B3**
SUB	`SUB rd, rs1, rs2`	rd = rs1 − rs2	000	0100000	ALU_SUB	`SUB x5, x6, x7`	**407302B3**
SLL	`SLL rd, rs1, rs2`	rd = rs1 << rs2[4:0]	001	0000000	ALU_SLL	`SLL x5, x6, x7`	**007312B3**
SLT	`SLT rd, rs1, rs2`	rd = (rs1 $<_s$ rs2) ? 1 : 0	010	0000000	ALU_SLT	`SLT x5, x6, x7`	**007322B3**
SLTU	`SLTU rd, rs1, rs2`	rd = (rs1 $<_u$ rs2) ? 1 : 0	011	0000000	ALU_SLTU	`SLTU x5, x6, x7`	**007332B3**
XOR	`XOR rd, rs1, rs2`	rd = rs1 ^ rs2	100	0000000	ALU_XOR	`XOR x5, x6, x7`	**007342B3**
SRL	`SRL rd, rs1, rs2`	rd = rs1 >> rs2[4:0] (zeros in)	101	0000000	ALU_SRL	`SRL x5, x6, x7`	**007352B3**
SRA	`SRA rd, rs1, rs2`	rd = rs1 >>> rs2[4:0] (sign in)	101	0100000	ALU_SRA	`SRA x5, x6, x7`	**407352B3**
OR	`OR rd, rs1, rs2`	rd = rs1 | rs2	110	0000000	ALU_OR	`OR x5, x6, x7`	**007362B3**
AND	`AND rd, rs1, rs2`	rd = rs1 & rs2	111	0000000	ALU_AND	`AND x5, x6, x7`	**007372B3**

alu_op = {instr[30], funct3}: the ALU code is the instruction's own bits.

verified by tb_cpu (test_ri), tb_alu

I Register–immediate

opcode **0010011** · OP_IMM

`alu_src 1` `alu_a_sel 0` `wb_sel 0` `reg_we 1` `mem_we 0` `branch 0` `jump 0` `jalr 0`

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
ADDI	`ADDI rd, rs1, imm`	rd = rs1 + imm	000	—	ALU_ADD	`ADDI x5, x6, -4`	**FFC30293**
SLTI	`SLTI rd, rs1, imm`	rd = (rs1 $<_s$ imm) ? 1 : 0	010	—	ALU_SLT	`SLTI x5, x6, 10`	**00A32293**
SLTIU	`SLTIU rd, rs1, imm`	rd = (rs1 $<_u$ imm) ? 1 : 0	011	—	ALU_SLTU	`SLTIU x5, x6, 10`	**00A33293**
XORI	`XORI rd, rs1, imm`	rd = rs1 ^ imm	100	—	ALU_XOR	`XORI x5, x6, -1`	**FFF34293**
ORI	`ORI rd, rs1, imm`	rd = rs1 | imm	110	—	ALU_OR	`ORI x5, x6, 0xF0`	**0F036293**
ANDI	`ANDI rd, rs1, imm`	rd = rs1 & imm	111	—	ALU_AND	`ANDI x5, x6, 0xFF`	**0FF37293**
SLLI	`SLLI rd, rs1, shamt`	rd = rs1 << shamt	001	0000000	ALU_SLL	`SLLI x5, x6, 4`	**00431293**
SRLI	`SRLI rd, rs1, shamt`	rd = rs1 >> shamt (zeros in)	101	0000000	ALU_SRL	`SRLI x5, x6, 4`	**00435293**
SRAI	`SRAI rd, rs1, shamt`	rd = rs1 >>> shamt (sign in)	101	0100000	ALU_SRA	`SRAI x5, x6, 4`	**40435293**

alu_op = {1'b0, funct3}, except funct3 = 101 (shift right) which keeps instr[30]. Without that rule a negative ADDI (imm bit 10 = 1 → instr[30] = 1) would silently become SUB.

verified by tb_cpu (test_ri)

Memory, control flow and constants

I Loads

opcode **0000011** · OP_LOAD

`alu_op` ALU_ADD · `alu_src` 1 · `alu_a_sel` 0 · `wb_sel` 1 · `reg_we` 1 · `mem_we` 0 · `branch` 0 · `jump` 0 · `jalr` 0

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
LB	`LB rd, imm(rs1)`	rd = sext(M[rs1+imm][7:0])	000	–	ALU_ADD	`LB x5, 3(x6)`	**00330283**
LH	`LH rd, imm(rs1)`	rd = sext(M[rs1+imm][15:0])	001	–	ALU_ADD	`LH x5, 2(x6)`	**00231283**
LW	`LW rd, imm(rs1)`	rd = M[rs1+imm][31:0]	010	–	ALU_ADD	`LW x5, 8(x6)`	**00832283**
LBU	`LBU rd, imm(rs1)`	rd = zext(M[rs1+imm][7:0])	100	–	ALU_ADD	`LBU x5, 3(x6)`	**00334283**
LHU	`LHU rd, imm(rs1)`	rd = zext(M[rs1+imm][15:0])	101	–	ALU_ADD	`LHU x5, 2(x6)`	**00235283**

The ALU computes the byte address rs1 + imm. funct3 bit 2 = unsigned, bits 1:0 = size. Accesses must stay inside one aligned word.

verified by tb_cpu_ldst (test_ldst), tb_cpu_mem

S Stores

opcode **0100011** · OP_STORE

`alu_op` ALU_ADD · `alu_src` 1 · `alu_a_sel` 0 · `wb_sel` – · `reg_we` 0 · `mem_we` 1 · `branch` 0 · `jump` 0 · `jalr` 0

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
SB	`SB rs2, imm(rs1)`	M[rs1+imm][7:0] = rs2[7:0]	000	–	ALU_ADD	`SB x5, 3(x6)`	**005301A3**
SH	`SH rs2, imm(rs1)`	M[rs1+imm][15:0] = rs2[15:0]	001	–	ALU_ADD	`SH x5, 2(x6)`	**00531123**
SW	`SW rs2, imm(rs1)`	M[rs1+imm][31:0] = rs2	010	–	ALU_ADD	`SW x5, 8(x6)`	**00532423**

rs2's low byte / half is replicated to every lane; the byte enables (be) pick the lane by addr[1:0].

verified by tb_cpu_ldst (test_ldst), tb_cpu_mem

B Branches

opcode **1100011** · OP_BRANCH

`alu_op` ALU_SUB · `alu_src` 0 · `alu_a_sel` 0 · `wb_sel` – · `reg_we` 0 · `mem_we` 0 · `branch` 1 · `jump` 0 · `jalr` 0

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	BR_COND	EXAMPLE	MACHINE CODE
BEQ	`BEQ rs1, rs2, label`	if (rs1 == rs2) pc += imm	000	–	`alu_zero`	`BEQ x5, x6, -8`	**FE628CE3**
BNE	`BNE rs1, rs2, label`	if (rs1 != rs2) pc += imm	001	–	`~alu_zero`	`BNE x5, x6, -8`	**FE629CE3**
BLT	`BLT rs1, rs2, label`	if (rs1 $<_s$ rs2) pc += imm	100	–	`lt`	`BLT x5, x6, -8`	**FE62CCE3**
BGE	`BGE rs1, rs2, label`	if (rs1 $\geq_s$ rs2) pc += imm	101	–	`~lt`	`BGE x5, x6, -8`	**FE62DCE3**
BLTU	`BLTU rs1, rs2, label`	if (rs1 $<_u$ rs2) pc += imm	110	–	`ltu`	`BLTU x5, x6, -8`	**FE62ECE3**
BGEU	`BGEU rs1, rs2, label`	if (rs1 $\geq_u$ rs2) pc += imm	111	–	`~ltu`	`BGEU x5, x6, -8`	**FE62FCE3**

Target = pc + imm, relative to the branch itself (not pc + 4). take_branch = branch & br_cond, computed in cpu.sv from alu_zero, lt, ltu.

verified by tb_cpu_sum (BNE), tb_cpu_br (BLT/BLTU/BGE/BGEU)

Jumps and upper immediates

J Jump and link

opcode **1101111** · OP_JAL

`alu_op –` `alu_src –` `alu_a_sel –` `wb_sel 2` `reg_we 1` `mem_we 0` `branch 0` `jump 1` `jalr 0`

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
JAL	`JAL rd, label`	rd = pc + 4; pc += imm	–	–	–	`JAL x1, 16`	**010000EF**

pc + imm goes to the PC (pc_target adder), pc + 4 goes to rd. The ALU is idle. JAL x0, label is a plain jump; done: JAL x0, done halts a test program.

verified by tb_cpu_call (test_call)

I Jump and link register

opcode **1100111** · OP_JALR

`alu_op ALU_ADD` `alu_src 1` `alu_a_sel 0` `wb_sel 2` `reg_we 1` `mem_we 0` `branch 0` `jump 0` `jalr 1`

INSTR	ASSEMBLY	OPERATION	FUNCT3	FUNCT7	ALU_OP	EXAMPLE	MACHINE CODE
JALR	`JALR rd, imm(rs1)`	rd = pc + 4; pc = (rs1 + imm) & ~1	000	–	ALU_ADD	`JALR x0, 0(x1)`	**00008067**

The ALU computes rs1 + imm; bit 0 is cleared before it reaches the PC. JALR x0, 0(ra) is the function return (ret).

verified by tb_cpu_call (test_call)

U Upper immediate

opcodes OP_LUI / OP_AUIPC

`alu_op ALU_ADD` `alu_src 1` `wb_sel 0` `reg_we 1` `mem_we 0` `branch 0` `jump 0` `jalr 0`

INSTR	ASSEMBLY	OPERATION	OPCODE	ALU_A_SEL	ALU_OP	EXAMPLE	MACHINE CODE
LUI	`LUI rd, imm20`	rd = imm20 << 12	0110111	2 (zero)	ALU_ADD	`LUI x5, 0x12345`	**123452B7**
AUIPC	`AUIPC rd, imm20`	rd = pc + (imm20 << 12)	0010111	1 (pc_cur)	ALU_ADD	`AUIPC x6, 0x1`	**00001317**

Both reuse the ALU with an ADD; only the A input changes (alu_a_sel). LUI + ADDI together load any 32-bit constant (that is what the li pseudo does).

verified by tb_cpu_br (test_br)

Pseudo-instructions

expanded by rvasm.py

PSEUDO	BECOMES
nop	ADDI x0, x0, 0
mv rd, rs	ADDI rd, rs, 0
li rd, imm	ADDI, or LUI + ADDI
not rd, rs	XORI rd, rs, -1
neg rd, rs	SUB rd, x0, rs
j label	JAL x0, label
jal label	JAL ra, label
jr rs	JALR x0, 0(rs)
ret	JALR x0, 0(ra)

PSEUDO	BECOMES
seqz rd, rs	SLTIU rd, rs, 1
snez rd, rs	SLTU rd, x0, rs
sltz / sgtz	SLT with x0
beqz / bnez rs, l	BEQ / BNE rs, x0, l
bltz / bgez rs, l	BLT / BGE rs, x0, l
blez / bgtz rs, l	BGE / BLT x0, rs, l
bgt / ble a, b, l	BLT / BGE b, a, l
bgtu / bleu a, b, l	BLTU / BGEU b, a, l

Not implemented: FENCE, ECALL, EBREAK (decoded as NOP), CSR instructions, misaligned accesses and traps. Memories: imem and dmem are 256 words each, word index = addr[9:2].

Control unit

rtl/control.sv · values set per opcode · bold = active

OPCODE	ALU_OP	ALU_SRC	ALU_A_SEL	WB_SEL	REG_WE	MEM_WE	BRANCH	JUMP	JALR
R OP_R	**{b30,f3}**	0	0	0	**1**	0	0	0	0
I OP_IMM	**{0,f3}***	**1**	0	0	**1**	0	0	0	0
I OP_LOAD	ALU_ADD	**1**	0	**1**	**1**	0	0	0	0
S OP_STORE	ALU_ADD	**1**	0	–	0	**1**	0	0	0
B OP_BRANCH	**ALU_SUB**	0	0	–	0	0	**1**	0	0
J OP_JAL	–	–	–	**2**	**1**	0	0	**1**	0
I OP_JALR	ALU_ADD	**1**	0	**2**	**1**	0	0	0	**1**
U OP_LUI	ALU_ADD	**1**	**2**	0	**1**	0	0	0	0
U OP_AUIPC	ALU_ADD	**1**	**1**	0	**1**	0	0	0	0
other	ALU_ADD	0	0	0	0	0	0	0	0

* OP_IMM: {1'b0, funct3}, except funct3 = 101 → {instr[30], funct3}. "—" = don't care (output unused). "other" = safe NOP defaults at the top of always_comb.

Signals

SIGNAL	BITS	MEANING
alu_op	4	ALU operation (alu_op_t)
alu_src	1	ALU B: 0 = rs2_data, 1 = imm
alu_a_sel	2	ALU A: 0 = rs1_data, 1 = pc_cur, 2 = 0
wb_sel	2	rd source: 0 ALU, 1 load, 2 pc + 4
reg_we	1	Write rd on the clock edge (never x0)
mem_we	1	Write dmem on the clock edge, lanes = be
branch	1	Branch: take_branch = branch & br_cond
jump	1	JAL: next PC = pc_target
jalr	1	JALR: next PC = {alu_out[31:1], 1'b0}

ALU operations

alu_pkg::alu_op_t

NAME	CODE	RESULT
ALU_ADD	0000	a + b
ALU_SUB	1000	a − b
ALU_SLL	0001	a << b[4:0]
ALU_SLT	0010	a $<_s$ b
ALU_SLTU	0011	a $<_u$ b
ALU_XOR	0100	a ^ b
ALU_SRL	0101	a >> b[4:0]
ALU_SRA	1101	a >>> b[4:0]
ALU_OR	0110	a | b
ALU_AND	0111	a & b

zero = (result == 0). $<_s$ signed, $<_u$ unsigned.

Branch condition

br_cond

FUNCT3	INSTR	BR_COND
000	BEQ	alu_zero
001	BNE	~alu_zero
100	BLT	lt
101	BGE	~lt
110	BLTU	ltu
111	BGEU	~ltu
010/011	—	1'b0

Next PC

{jalr, jump|take}

SEL	WHEN	PC_NEXT
1 0	JALR	{alu_out[31:1], 1'b0}
0 1	JAL, taken branch	pc_target = pc + imm
else	everything else	pc_plus4

Memory lanes

addr[1:0]

SB @ addr[1:0] = 00 → be = 0001

[31:24] [23:16] [15:8] **[7:0]**

SB @ 11 → be = 1000

[31:24] [23:16] [15:8] [7:0]

SH @ addr[1] = 0 → be = 0011

[31:24] [23:16] **[15:8]** **[7:0]**

SH @ addr[1] = 1 → be = 1100

[31:24] **[23:16]** [15:8] [7:0]

Rule: register side = low bits, memory side = lane by addr[1:0].